GNN-IP Pair Execution
랩 세미나 최종 발표

류 , equivariant 추론 파이프라인 이해 , 관련 추론 최적화 연구 , 그리고 SevenNet 기준 실험 결과와 discussion

김민창 / DenseMLIP / 30 min seminar

오늘 발표 구성

30분

01 GNN-IP 와 equivariant 추론 파이프라인

02 재사용 가능한 연산값의 분류와 구현 포인트

03 연관 추론 최적화 연구와 현재 구현 위치

04 실험 결과 , profiling, discussion

GNN-IP 란 무엇인가

원자 구조를 그래프로 보고 total energy 를 예측한 뒤 force 와 stress 를 gradient 로 얻는 모델

핵심 입력은 원자 종류 , 상대 위치 , cutoff 안의 local neighborhood

에너지 자체는 rotation-invariant scalar 이고 , force 는 위치 미분으로 나온다

따라서 intermediate representation 은 equivariant 하게 유지하는 것이 유리하다

이 슬라이드는 작업자 외 학생들을 위한 1-2 장짜리 전체 설명 파트다 .

Equivariant GNN-IP 추론 파이프라인

Atoms / Cell / Neighbor list → Edge geometry (r, |r|, SH, basis) → Equivariant interaction blocks → Atomic energy readout → Force / Stress via gradient

핵심 포인트 : energy 는 node-level scalar readout 이지만 , 그 전에 대부분의 비용은 edge geometry 생성과 equivariant interaction block 에 들어간다 .

재사용 가능한 연산값의 분류

Pair-shared geometry

distance |r|

radial basis

cutoff

Parity-recoverable

reverse SH via (-1)^l

canonical pair direction

Direction-specific

source feature x_i / x_j

final message TP output

Node / global outputs

aggregation

atomic energy

force / stress

이 분류가 있어야 “무엇을 exact 하게 줄일 수 있고, 무엇은 줄일 수 없는가”를 구분할 수 있다.

SevenNet 현재 구현과 목표 구조

현재 구현

- pair metadata
- geometry / SH 일부 재사용
- weight_nn 일부 재사용
- directed TP
- node aggregation

현재는 pair-aware reuse 이지 , pair-major TP 는 아니다 .

목표 구조

- pair metadata
- pair-major geometry 유지
- pair-major TP or delayed expansion
- backend-aware reduction
- runtime policy selection

핵심은 pair layout 을 끝까지 유지하는 것이다 .

연관 추론 최적화 연구 : 이 발표에 필요한 부분만

Backend fusion

FlashTP

cuEquivariance / OEQ

TP kernel optimization

Runtime / graph

neighbor cache

graph layout

batching / memory traffic

System / distributed

ghost communication

schedule

runtime policy

이번 발표는 커널 최적화 자체보다 , pair-aware runtime 과 backend co-design 이 어디에 들어가는지를 설명하는 데 초점을 둔다 .

실험 질문과 평가 프로토콜

질문 1: 어떤 그래프에서 pair execution 이 실제로 이득인가 ?

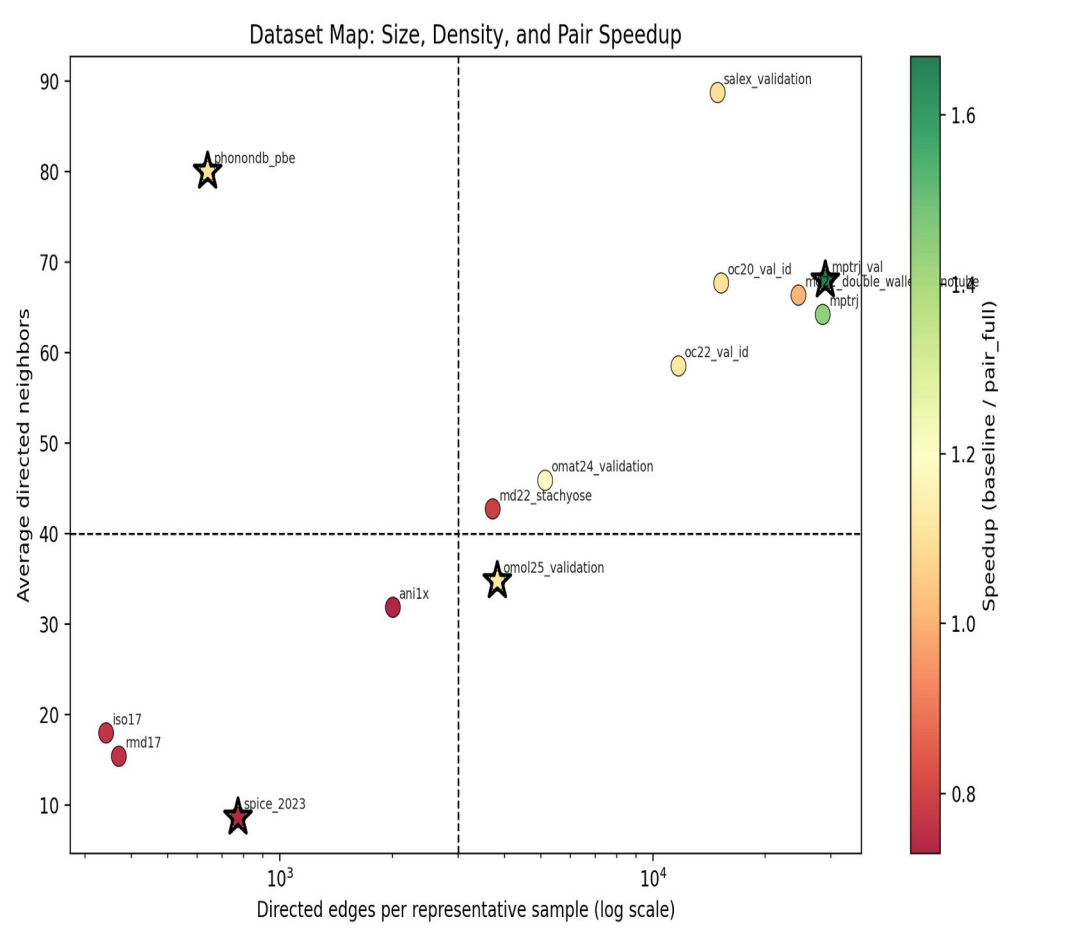
질문 2: 재사용되는 연산은 전체 시간에서 얼마나 큰 비중인가 ?

질문 3: 현재 구현은 exact pair-major execution 과 무엇이 다른가 ?

환경 : SevenNet / 7net-omni / single-GPU lab benchmark + public real dataset evaluation

속도 , speedup, profiling, GPU utilization, size-density 를 같이 봤다 .

데이터셋 size-density map



x 축 : 대표 샘플 directed edges

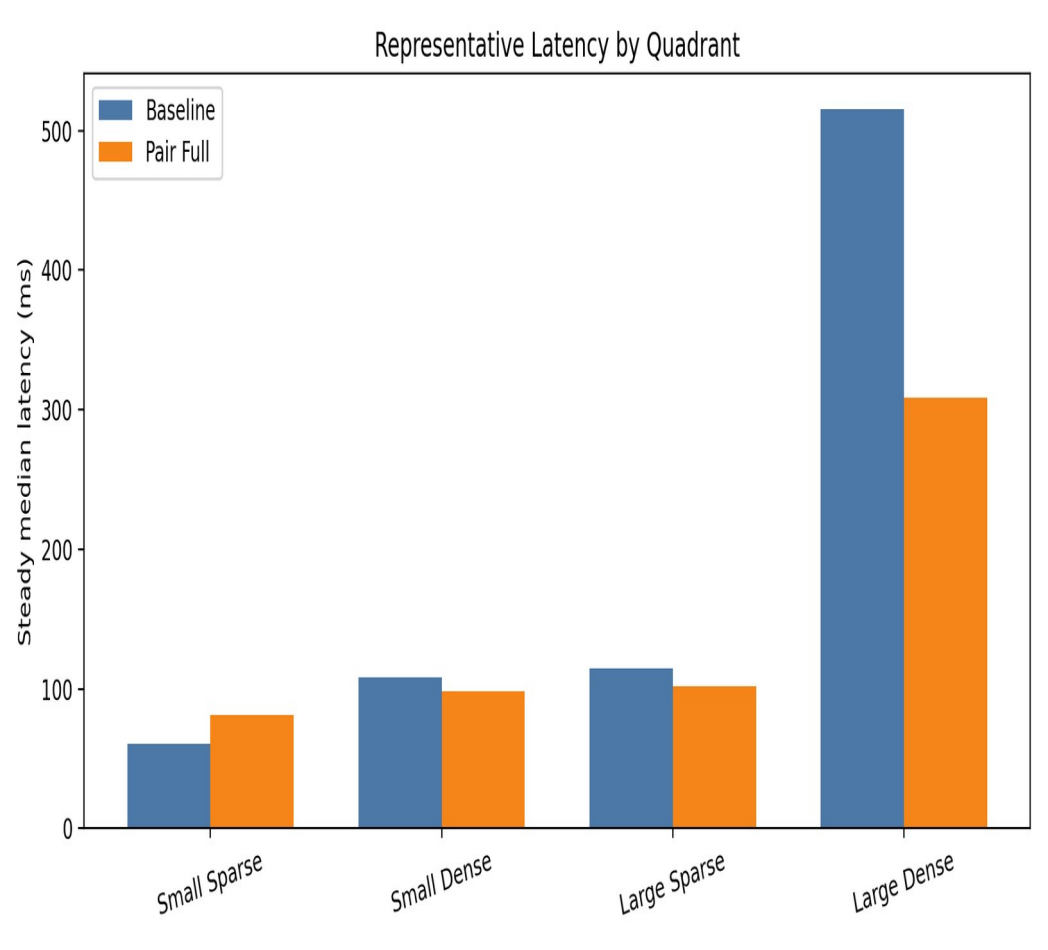
y 축 : 평균 directed neighbors

threshold: large = edges >= 3000, dense = avg neighbors >= 40

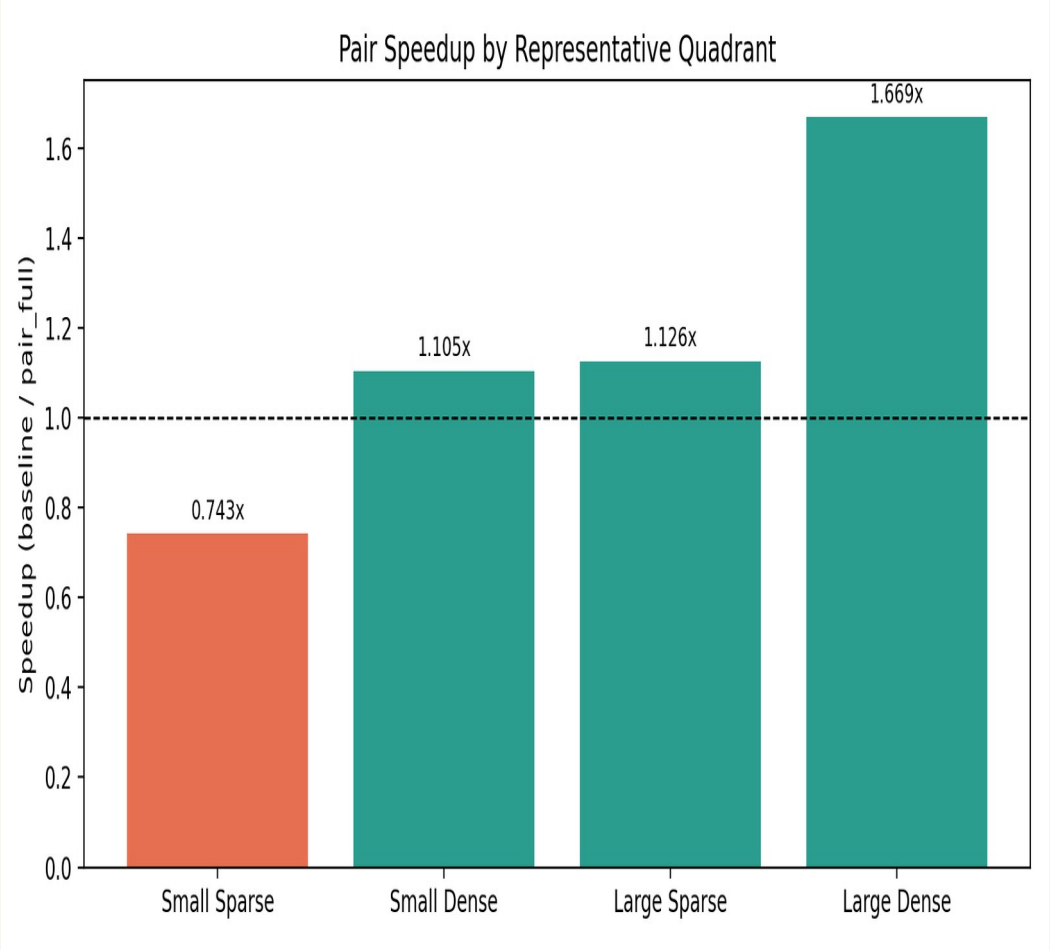
발표용으로 네 구역 representative 를 명시해 비교했다

네 구역 대표 비교

Latency comparison



Speedup comparison



Quadrant 결과의 핵심 해석

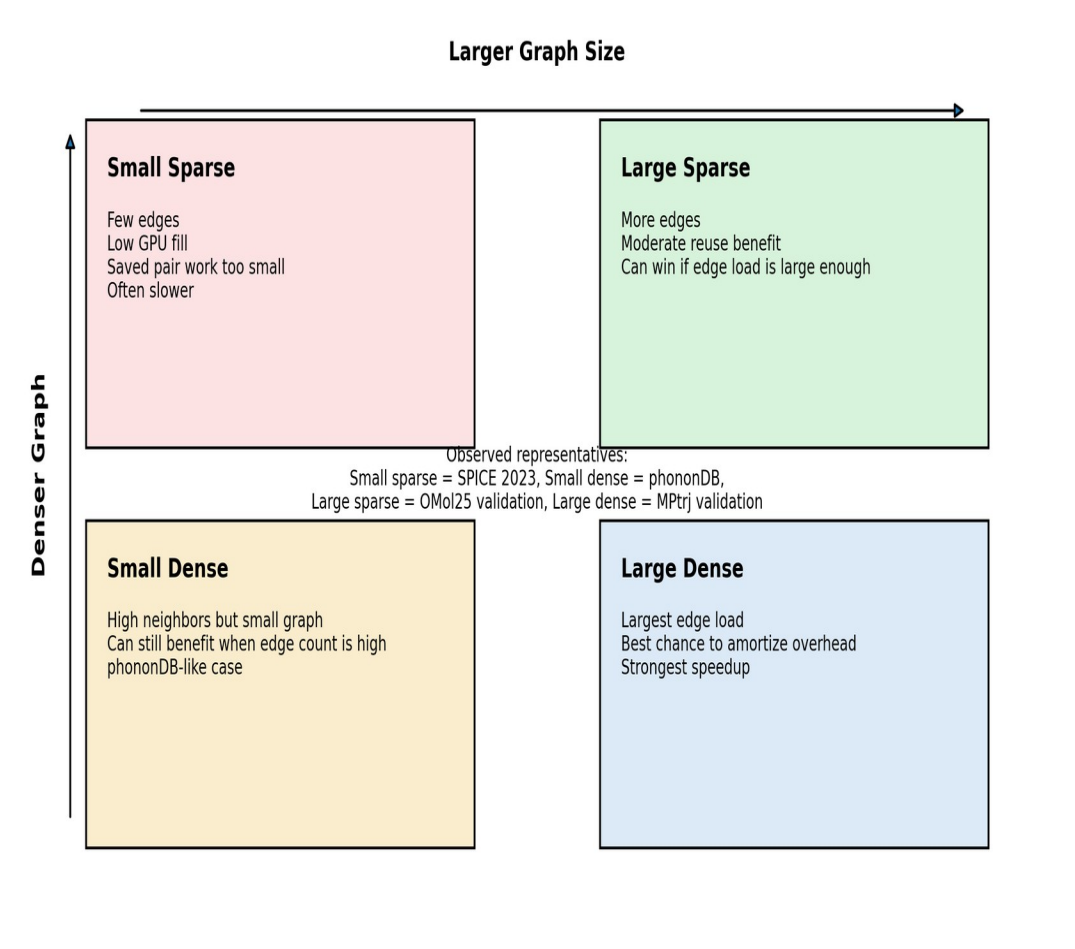
small sparse: SPICE 2023, 0.743x

small dense: phononDB, 1.105x

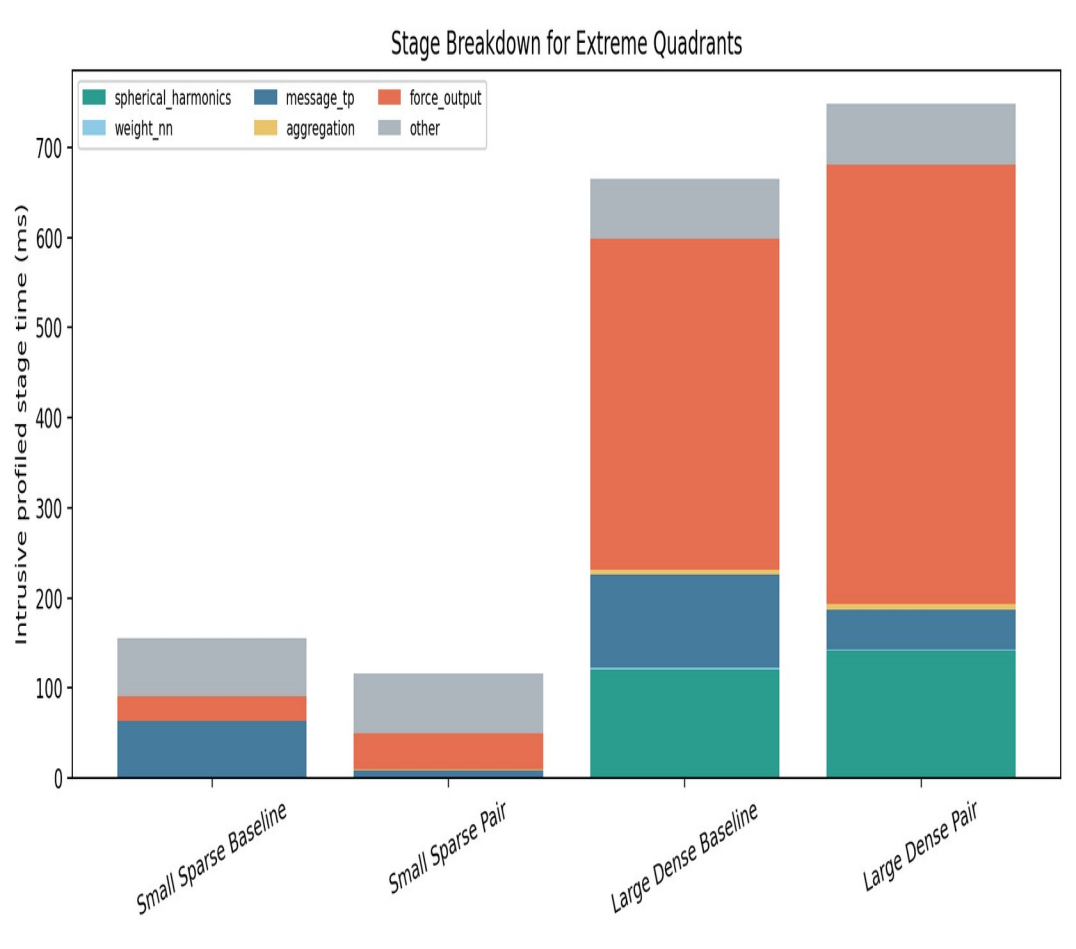
large sparse: OMol25 validation, 1.126x

large dense: MPtrj validation, 1.669x

즉 atom 수 단독이 아니라 edge load 와 density 조합이 핵심이다



Baseline 상세 프로파일 : extreme cases



small sparse baseline 은 TP 와 force_output 비중이 크다

large dense baseline 은 SH + TP + force_output 이 동시에 크다

weight_nn, gather, aggregation 은 baseline 에서도 상대적으로 작다

그래서 현재 pair execution 의 이득은 일부 구간에서만 크게 보인다

랩 실험 결과 요약

steady-state 기준 , dense-large 구간에서 speedup 이 가장 크고 dense/small or sparse/small 에서는 손해 또는 약한 이득

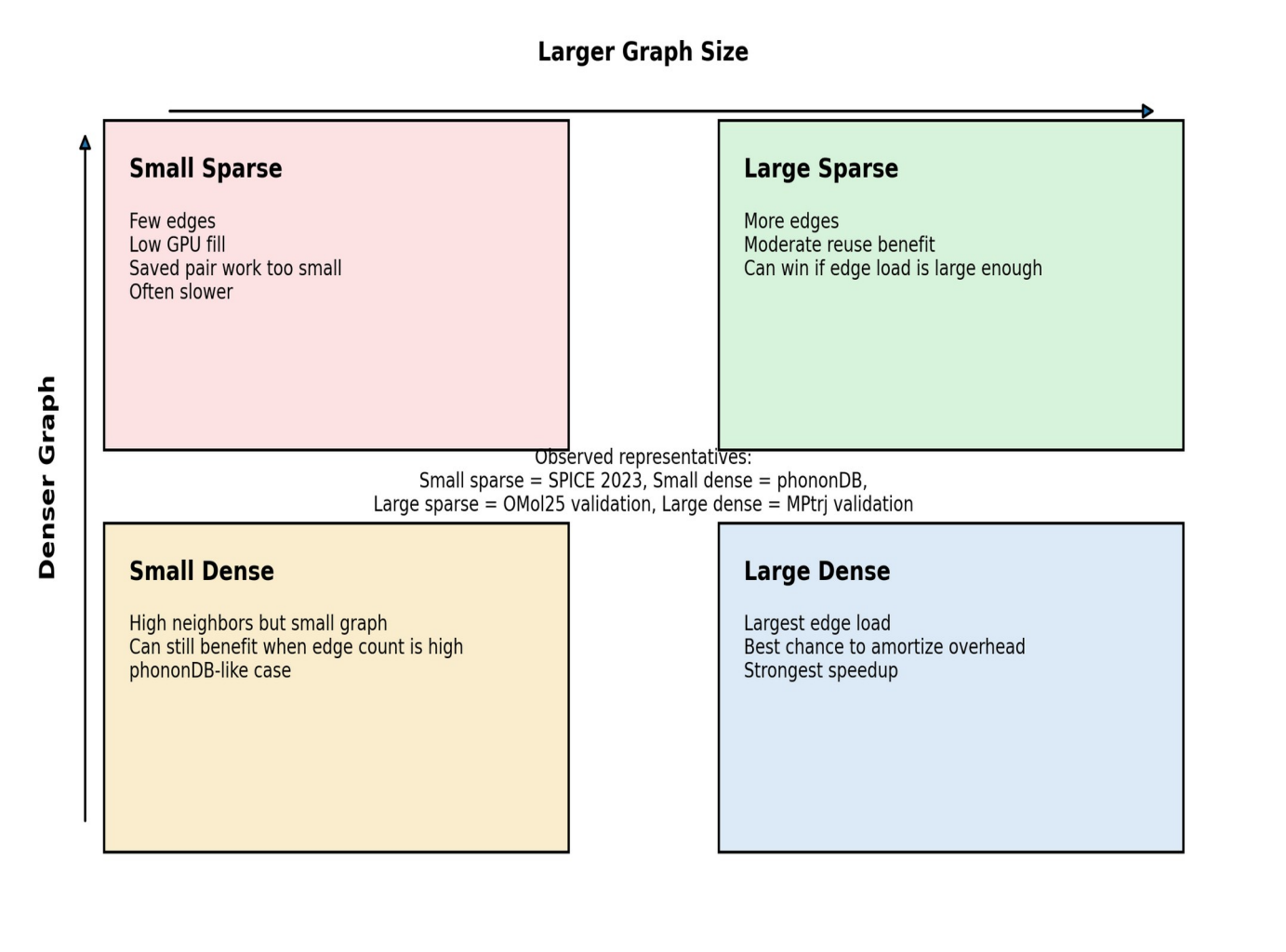
GPU utilization 과 stage profiling 을 같이 보면 , 작은 그래프는 saved work 절대량이 작고 under-fill 문제가 남는다

반대로 dense-large 에서는 geometry reuse 가 실제 wall time 감소로 이어진다

현재 구현은 pair-major TP 가 아니므로 speedup ceiling 이 존재한다

단일 GPU lab benchmark 와 public real dataset benchmark 를 함께 해석했다 .

Discussion: 왜 어떤 경우는 이기고 , 어떤 경우는 지는가



small sparse: saved work 절대량이 작고 GPU fill 도 낮다

small dense: atom 수는 작아도 edge load 가 충분하면 이길 수 있다

large sparse: 크기만 크다고 자동으로 이기진 않지만 break-even 을 넘기기 시작한다

large dense: geometry reuse 가 가장 크게 먹히는 구간이다

지금까지 구현한 것과 남은 것

완료

pair metadata / topology cache 정리

geometry + weight reuse 구현

baseline / pair 실험 파이프라인

size-density / profiling / GPU util 분석

세미나용 자원 및 다이어그램 생성

남은 핵심 과제

pair-major TP kernel

FlashTP 와의 진짜 backend co-design

distributed pair schedule

더 큰 public dataset 완전 확보

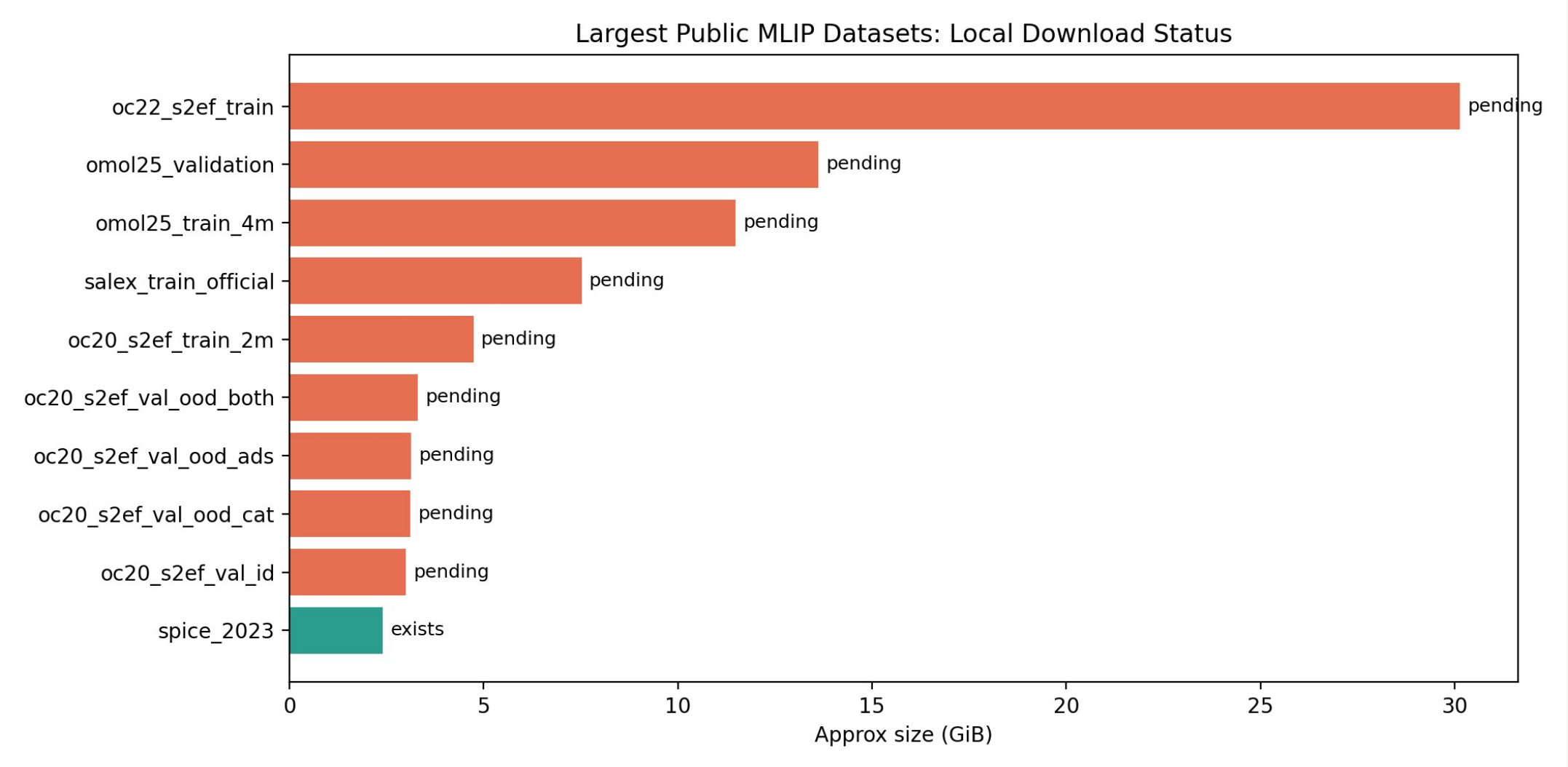
additive profiler 정리와 논문화

결론

재사용 연산값을 pair-shared / parity-recoverable / direction-specific / node-global 로 나누는 것이 출발점이다 .

현재 SevenNet 구현은 geometry/SH/weight reuse 까지는 왔지만 , pair-major TP 는 아직 아니다 .

실험상 성패는 결국 edge load 와 density 의 조합으로 갈리고 , 가장 강한 구간은 dense-large 이다 .

다음 단계의 연구 포인트는 pair-major execution 과 backend/runtime co-design 이다 .

Appendix: 큰 데이터셋 확보 상태